			Name:	**Jonathan Koral**			**Semester start:**	1/26/2026			
			Section:	**VC1A**							
			Total Hours:	243.56							

Week	Start	End	Supervisor discussion	Team discussion	Design	Coding	Documentation	Testing & Debugging	Research, Training, Learning	Other	Total Hours for the week
1	2026-	2026	0	0	0	0	6.5	0	7	4.4	17.9
2	2026-	2026	0	0	1.5	0	4	0	5.5	0	11
3	2026-	2026	0	0	4.25	0.1	4.6	0	6.6	5.5	21.05
4	2026-	2026	0	0	0	3.25	12.6	0	0.2	0	16.05
5	2026-	2026	0	0	0	9	0	0	7	0	16
6	2026-	2026	0	0	0	6	4.3	1	3.9	0	15.2
7	2026-	2026	0	0	0	9	0	5.5	1	3	18.5
8	2026-	2026	0	0	4	11	0	0	0	0	15
9	2026-	2026	0.01	0	0	0	0	6	0	9	15.01
10	2026-	2026	0	0	0	7.5	0	7	2	0	16.5
11	2026-	2026	0	0	10	0	0	0	4.5	1.35	15.85
12	2026-	2026	0	0	0	8	2	6	0	0	16
13	2026-	2026	0	0	0	16	0	0	0	0	16
14	2026-	2026	0	0	7.5	11	0	0	0	0	18.5
15	2026-	2026	0	0	0	8.5	3	0	0	3.5	15
		Total	0.01	0	27.25	89.35	37	25.5	37.7	26.75	243.56

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
1/26/26	1.4	Research, Training, Learn	Went on brightspace for the CISC. 4900 Spring course and went over the "About the Course" webpage. After that, I went over all of the orientation slides and took my time understanding my respnsibilities and what I have to accomplish this semester. I went over the advice on keeping time logs webpage and decided to use the template provided.		
1/26/26	1.25	Other	Started filling out the "Project Details" google form due the second of feburary.	Had to think hard about the questions that are being asked and reflect on my project as a whole.	Only got up to the question about "solution-space" before decidingto take a break.
1/27/26	1.5	Other	Continued where I left off in the "Project Details" google form. I wrote down the solution-space, implementation-space, the key action steps, implementation-space, its executable form, and the brief explanation of the value-add of my project.		
1/27/26	2.5	Research, Training, Learn	I went on to review mny old notes about CPU scheduling algorithms. I started by reading the notes about FCFS, SJF, SRTF, RR, Priority, and CFS. After I refreshed my memory, I did some practice writing down gantt charts for all algorithms with different processes. I did this because I wanted to make sure I know how to do what my projhect is supposed to do with my own hands on pen and paper.	Reviewing old notes and problems along with new problems.	My mind is now refreshed and I am ready to start the first steps of the project.
1/27/26	2	Research, Training, Learn	I was browing the web looking at resources about the software development life cycle. I had a general idea about what it is but I didnt know how to write it down on a document. Since im doing a solo project without any organization, I had to ignore stuff that related to that. There was a lot of useful information that I learned.	I never wrote documentaion using the stages of the SDLC before so I was stuck on what to do next.	I learned a lot of useful information about the SDLC from resources like https://www.geeksforgeeks.org/ and others that had more detailed examples about each stage.
1/28/26	0.6	Research, Training, Learn	I had to refer back to online resources to refresh my memory about the first stage of the SDLC which is planning.	Needing to review information found the day before.	Wish I could remember everything from the other day but small details needed to be refreshed.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
1/28/26	3.5	Documentation	I started writing down my documentation for the SDLC. I was writing down the first stage which is "Planning". I started by writing down the project overview which is there to explain what the project is about. I wrote about how this CPU scheduling algorithm simulator is a web application that will help visualize the different scheduling algorithms such as FCFS, SJF, SRTF, RR, and Priority. I also explained the goal of the project. Next I wrote down the Problem Statement section where I wrote about how students may struggle verifying if their gantt chart is correct and how students who do this for the first time might not know they are making mistakes or what they are doing incorrectly. I also explained the different resources students can use but this may make them overwhelmed and confused with all the different approaches of each algorithm. After that I went on the write down the end users section which explains the end users for my project which will be individuals who are learning CPU scheduling algorithms and want a fast and simple way to experiment with them and that another user could be other people who need a visual representation of cpu schedulers for academic purposes. I ended the day with writing down the project goals with basic things I should get completed for the project to work smoothly.	Getting used to writing organized documentation.	I learned a lot about my project doing this. I got a better picture of what i want to build. This was very helpful.
1/29/26	0.5	Research, Training, Learn	Since im moving onto the next step of the SDLC which is the requirements, I had to get a better idea of what I need to write about before I start writing.		

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
1/30/26	3	Documentation	I went back to my document and started the second stage of the SDLC which is the requirements section. I started out by writing down the fucntional requirements. I wrote down the input for the project should be which is the user should select a algorithm from a list that is provided. The user can create processes with arrival times and burst times, and when both of these inputs are filled out, they are allowed to submit the data for the simulation to start. Next was the output for the project which is basically, displaying a gantt chart, showing the average waiting time, turnaround time, and throughput, and a table that shows the processes and all the information associated with them. Then I wrote about the backend which will use REST API's to take and process the data, and the APIU needs to run the scheduling algorithm that the user selected from the processes that were provided. After that i moved onto the Non-functional requirements which are that the user interface has to be simple and easy to use, the web application should be able to run on both desktop and mobile devices, the user shouldnt have to wait a long time after they submit, and that the algorithms are implemented correcly and there should be no mistakes in the output and it should always be consistent. I finished it off with a Conditions section which states that there will be a seperate frontend and backend that communicate with eachother using REST API calls. The application needs to be used in a web browser and ther shouldnt be a database because we are not collecting information from the user. I want there to be no forced signup because the user shouldnt have to sopend time creating an account just to use the CPU scheduling algorithm. I think that would be a bother to the user and an account wouldnt benefit them in any way.	Reflecting on what is required for the user to input for ther simulator to be able to create a gannt chart with other relevant data.	There is a more clear path now that requirements are written.
2/1/26	0.15	Other	I completed the orientation quiz and got 5.0/5.0 on the first try.		
2/1/26	1.5	Other	I had to rewrite my resume so I can submit it to brightspace.		
2/3/26	2.5	Research, Training, Learn	I read articles and examples on REST API design. I needed to review my knowledge of REST API so I have confidence when I get to coding. I also want to be following best practices like naming and using the appropriate http methods and status codes.	There was a lot of material that I had to review	I might have to refer back to this source https: //restfulapi.net/ when I get to coding so I follow best practices.
2/4/26	3	Research, Training, Learn	Because my project requires I make a Gantt chart in the frontend to show the results, I had to do some research in how to code them in the frontend. There is a lot that goes into it when writing it in JavaScript but it seems very straightforward.	There was a lot of different resources to look at for building Gantt charts and then i realizedthat they aren't exactly what I thought they were. After a while, I found a resource that I liked and then I got overwhelmed by the amount of code that goes into writing Gantt charts in JavaScript.	Once I got to reading the raw JavaScript, I saw that its not as scary as I originally thought. For my project, im going to need to customize it differently than what the tutorial showed so its not a straightforward guide to my special project which is good for my learning. The resource I used was https: //www.google.com/amp/s/www.fusioncharts. com/blog/5-ways-to-build-gantt-charts-in-javascript/amp/

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/5/26	2.5	Documentation	I decided to continue writing my SDLC. I started writing stage 3 which is "System Architecture Design". I wrote that the application will be a client-server architecture. The frontend will accept user input, and display charts and results. The backend will handel the business logic through REST API calls. For the frontend design, I wrote down what needs to be accepted as input and the way the frontend will ask the user for it. With the backend, I stated that each algorithm will be implemented in separate class to be organized and modular.	I was busy today but I found time to work on the project.	There are a lot of thing I had to thing about such as how the user will interact with the website, how the frontend and backend connect, and how the backend should be designed.
2/7/26	1.5	Documentation	I worked on the next stage of the SDLC which is "Interface Design". I wrote down what I wanted from the user interface. I want some sort of box or area where the user can input their processes. I also want to add a drop-down menu that lists all of the scheduling algorithms that users can pick from. For results, I want a chart that lists the processes and relevant information about them, and a chart that visualizes the algorithm with the processes.		This is a good step for getting ideas out of my head. The next step is to draw a examples of how I want the user interface to look.
2/8/26	1.5	Design	I got blank sheets of copy paper and started drawing out how I want the web page to look like. I have the a rough design for what it would look like when you first arrive to the website. I have a different page for evey function that the user experiences. One for picking the algorithm, one for adding processes and also the visual representation of the processes that the user adds. There is also a page for the results where the user can check out all of the calculations and results.		Drawing on paper helps me organize my thoughts of how I envision the design.
2/9/26	4	Other	I continued the project proposal survey. I had to answer a bunch of questions regarding the project. There was also a section that took me a while to finish. I had to write down a projected PoC, Prototype, Pilot, MVP, and MDP.	I had to so research to find out what a PoC, Prototype, Pilot, MVP, and MDP. I feel like these questions were very repetitive because what I wrote for the PoC, is also similar to what the prototype has but with a little more usability. then the MVP is the same but except all the Algorithms are implemented.	There is a lot of writing you have to do before you even get to coding.
2/11/26	0.1	Coding	I set up my github account and created a repository for the project. I also added a file called project-tracker.md		
2/11/26	1.5	Other	I reviewed and edited all of the short responses in the project proposal survey. I made changes where I saw fit and added more details when it looked like more could be added.	Fixing grammar and finalizing details for each response.	I found that the earlier versions of my responses didn't fully answer the question that was asked but I had already written a lot so I only needed to make minor changes.
2/12/26	1.25	Documentation	I started working on the 1st draft for my slides. After reading the description, I had a good idea of what to start writing down. After writing down the project title and contact information, I had to write the elevator pitch. Because I wrote down a SDLC document, I had a good idea for what I should write for the elevator pitch. It was a combination of my "Project Overview", and my "Problem Statement" along with my goal for this project.		

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/12/26	1.1	Research, Training, Learn	I did some research on how to write a elevator pitch. The idea of an elevator pitch is to explain your project in under a minute.	The resources I used taught me how to write an elevator pitch but it was mostly referencing to an elevator pitch you would make about yourself which gave me a hard time trying to translate it for a project. For example, an introduction about myself dosen't sound right for this type of project so I decided to quickly introduce my project.	The resource that I decided to stick to is the grammarly article on how to create an elevator pitch. https://www.grammarly.com/blog/business-writing/elevator-pitch/?msockid=2b1d19563324661c20230a8332f0678b
2/12/26	0.2	Documentation	The next step was to list out the tools I will use for this project. For software, I will use Java 25, Spring Boot 4.0.0, IntelliJ, Github, HTML, CSS, Javascript. I also will make use of testing usin Postman, JUnit, and use a web browser for testing the user interface.	Now that I have the first four slides done, I have to do some research on how what diagrams I should make for my project.	
2/13/26	4.5	Research, Training, Learn	For the diagrams section, I didnt just want to have two small diagrams, I wanted multiple diagrams that cover different parts of the project. I had to do research different what type of diagrams there are for software documentation. The ones I came across and decided were a good fit to inculde in the sldies were the System Architecture, Data Flow, Wireframe, Class, and Activity diagrams. Having a System Architecture diagram is good for my project because I can display what the system is made of. The Data Flow diagram shows how the data will move through my system. The Wireframe shows what the user will see and what interactions will look like for the user. The Class diagram shows what the bakcend will look like in terms of the classes and what they will consist of. The Activity diagram will show the workflow of the system step by step from the users input to the systems output.	Having all of this information, I am ready to start creating the five diagrams.	There are a lot of planning that goes into building software and creating diagrams is a good way to visualize what you want. Having your system documented and written down is one thing but creating diagrams allows you to look back and visualize what the system will look like.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/13/26	4.25	Design	First I started drawing out the System Architecture which inludes the web browser user interface, REST API endpoints, the Scheduling Algorithm implementations for FCFS, SJF, SRTF, RR, and Priority. There is also a Backend output that the fortend can use to create the Gantt chart, process table, average waiting/turnaround time, and throughput. Now with the Data Flow diagram, it shows how the User input travels from the frontend to the API, to the Algorithm, then to JSON for the frontend to read, and then the data gets used to create the results. The wireframe was my favorite part to make. I included all the basic designs that is neccessary for this project. There is a "add process" button that allows the user to add processes, the processes can be removed after they are added, you can select multiple algorithms, you can run the simulation, and the results will be displayed on the screen. This was a very basic design but I do not think this will be the final design. For now, its the minimum design that makes it fully functional. After that, I created the Class diagram. I sketched out the different classes that I think I will need and their attributes. The classes I have so far are the ScheduleRequest, ScheduleResult, ScheduleService, Process, GanttEntry, and the ProcessResult class. I also made a AlgorithmStrategy interface that will be implemented into FCFS, SJF, SRTF, RR, and Priority. The last diagram I made was the Activity diagram. We start out with the Enter Processes block where the user enters their processes, then it gets checked to see if the input is valid. If the input is valid, we can select an algorithm, if not we have to enter the process correctly. When we select an algorithm, if RR is selected, then we have to enter a quantum value, otherwise we don't have to. Then we can run the simulation and the output will be generated.	I had problems trying to draw the wireframe inside a software tool. I decided that drawing on paper would be more efficient and I will be able to better express my ideas.	A lot of work goes into making diagrams and it requires you to know a lot about your project before you make them. Because I wrote a lot of documentation already, I didnt get stuck on any of the diagrams when making them.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/14/26	1.15	Documentation	The next step was to create a tentative schedule. I started the rest of the schedule at week 5 which is when I will set up the backend. Week 6 will be the first implementation of a CPU scheduling algorithm which will be FCFS. I will also create the Demo Recording video that week. Week 7 will be the week I create the SJF implementation. Week 8 will be when I start setting up the frontend so It can be tested by users. Week 9 will be the implementation of the SRTF algorithm. Week 10 will be to focus on the frontend and make it more user friendly and make sure the format for the output is displayed in the proper way. Week 11 will be when I finish up the full draft for the slides. Week 12 will be the implementation of Round Robin and it will have a focus on making the frontend accept the quantum time as input when Round Robin is selected. Week 13, I will have a strong base for my project where I can make a confident demo recording video. I will also implement Priority Scheduling, but whether it will be implented before or after the demo video will be decided on that week. Week 14 will have a strong incentive to strengthen the frontend and make it the most presentable as possible. The user interface wilkl be updated, the layout will change to a stronger one, and any bugs found will be fixed. There will also be error messages that will pop up instead of basic red text. Week 15 is the final week and I will finish up the final slides and create the final demo video and submit the project to be graded.	I decided not to make a data source slide because I decided that my project will not be using any data or storing any data anywhere because it is not neccessary for this project.	I will put a heavy focus on the frontend because I do not like basic and raw user interfaces like some other websites have.
2/14/26	1	Research, Training, Lear	I did some reserch on how to create Use Cases. The basics are to describe the system, identify the actors, define their goals, create a scenario, and to consider alternate flows.	Next steps are to create the three use cases.	I referenced an artice that I found on the internet. https://www.figma.com/resource-library/what-is-a-use-case/
2/14/26	2	Documentation	The first use case I made was a student who needs to verify their homework solutions, the student enters their processes and runs the simulation, and they receive the output. For the next use cases, I remembered that after writing a successful scenario, i have to write alternate flows that lead to different outcomes. For my project, that would be the use of algorithms like Round Robin that requires quantum time and Priority scheduling which requires processes to have priority. For use case 2, the user wants to understand how different algorithms behave when they are given the same input. the user enters the processes and selects the algorithms FCFS, SRTF, RR (Q=2) and runs the simulation. The user can now compare the algorithms side by side and see how they respond to the same processes. For use case 3, its an instructor who wants to show the class how priority scheduling works. The instructor enters the processses along with priority. The instructor selects Priority scheduling and runs the simulation. The instructor can now show the class how priority scheudling works and a can express te advantages and disadvantages in real time.		Doing research on making Use Cases helped me out because originally in muy SDLC document, I had a form of use cases but they we not tailored to be considering alternative flows. Because I did research, I was able to write use cases that dont shows the same thing.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/17/26	0.2	Research, Training, Learn	I wanted to create a Software requirements Specification document (SRS) so I looked on the internet and found a template I can follow that is ieee standard.	The next step is to use this template as a guide to create the document	
2/17/26	1.5	Documentation	I stated out by writing the cover and index. The first thing I had to write down was the Introduction. This section is about telling the reader the purpose of the SRS for the project. It goes over that it is a web application designed to help students visualize and understand CPU scheduling algorithms. It will cover FCFS, SJF, SRTF, RR, and Priority. I also go over the intended reading audience and the product scope so the reader gets a better idea of what this porject is about. I also included references that helped me create this document.	Next steps are to write down the Overall Description of the project.	
2/17/26	1.5	Documentation	The next part I wrote down is the overall description of the project. This section included the Product Perspective, Product Functions, User Classes, Opating Environments, Design and Implementation Constraints, and User Documentation. The Product perspectiveve focuses on telling the reader that this project uses a client-server architecture that has a frontend that collects the users input, sends the data to the backend through an API, the backend runs the algorithm, formats the results in JSON, sends it back to the frontend, and creates the gantt chart, process table, average waiting/turnaround time, and throughput. The product functions are that the user can enter processes with relevant attributes, select algorithms, and after the user runs the program, the backend generates all the relevant data the user needs and makes sure all inputs are valid. For design and implementation constraints, I wrote that the system has to use java and springboot for backend logic, there must be no database, and when round robin is selected, give the user the option to input time quantum.	The next step is to write down the External Interface Requirements	
2/18/26	1.2	Documentation	I wrote down the External Interface Requirements which has sections for user interface, hardaware interfce, software interfaces, and communications interfaces. The User Interface section has my wireframes that I drew out so the reader can get a visual representation of how the user interface will look. For the software interfaces section, I explained that this project uses Spring Boot for the purpose of providing REST API endpoints, handling JSON, and managing dependency injection for cpu scheduling algorithms. All communication between the frontend and the backend will be from the use of JSON. There will be classes for the backend like Process, ScheduleRequest, ScheduleResult, GanttBlock, and ProcessResult. The external libraries that will be used are Lombok, Spring Starter Web, and Spring Starter Validation.	The section that is next is the System features which will go over in more detail what the system will provide.	After getting down the first three chapters, I am feeling confident in the quality of my documentation.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/18/26	1.5	Documentation	I started writing down the System features.The First system feature I wrote down was the Input Processes feature. This feature allows users to enter a list of processes with custom attributes that will be used in the simulation. I ranked this feature as high priority because without it, the user cannot run custom processes for all simulations. I wrote down a few stimulus/response sequence between the user and system. The functional requirements for this feature is that the program has to allow the user to input a list of professes, each process must include arrival time, burst time, and priority. The program has to evaluate the arrival time and burst times are positive numbers and the user cannot run a simulation if the processes are empty. There should also be error messages for invaluid inputs.		I did by best to try and follow the format of the example docuyment I was referencing.
2/18/26	1.2	Documentation	The next system feature is the algorithm selection feature. This feature allows users to select what CPU scheduling algorithm they want to simulate. It's a high priority feature because without being able to select an algorithm, the program won't be able to run a simulation. The functional requirements include that the program has to allow the user to select their desired algoithm, the program must have FCFS, SJF, SRTF, RR, and Priority scheduling algorithms, if RR is selected, the program must ask for a quantum value, the program has to validate that the quantum value is positive, and that the program has to hide the quantium option fo all algorithgms besides RR.		
2/19/26	1.2	Documentation	The next system feature is a simulation feature. This feature runs the scheduling algorithm with all the processes as inputs. It's a high priority feature because it provides the results needed to make the Gantt chart, process table, average waiting/turnaround times, and throughput. The functional requirements are that the program has to validate all the inputs before running the simulation, the program must send a JSON request that has the algorithm and processes, the backend must run the scheduling algoirthm that was chosen with the processes that were added, and the backend must return the results to the frontend.		
2/19/26	1.5	Documentation	The next system feature is the gantt chart vizualization. This feature displays the cpu scheduling algorithm gantt chart to the user. It's a high priority feature because this is the whole reason the user would want to use this project for. To visualize the cpu scheduling algorithm of their choice. Lets say that a user runs the simulation, the backend should return the results and the frontend has to organize the results and create a nice visualization for the user to look at and understand. The functional requirements are that the program must use the backend results to create a gantt chart, each gasnt block must shows the process with the start and end times, and the gantt chart should be easy to look at with different colors for each process.		

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/20/26	1.5	Documentation	The next system feature is the process table and relevant data. This feature displays the processes table after execution along with the average waiting/turnaround time and throughput. It's a high priority feature because users rely on this data to verify their work relating to cpu scheduling algorithms. The functional requirements are that the program must display the waiting and turnaround times for each process, the program must calculate and display the average waiting and turnaround times, the program must calculate the throughput, and the program should display the results in a table.		
2/20/26	1	Documentation	The next system feature validation. This feature makes sure that the system runs as expected by making sure the data the user entered is valid. It's a high priority because making sure that the algorithm functions properly when run is important for my project's reputation. I don't want to have a project that claims to simulate something but ends up doing it incorrectly. The functional requirements are that the program must be able to detect invalid inputs, the program must prevent the user from starting a simulation when their inputs are invalid, the program should provide clear error messages explaining the problem, and the program should visually show the field is invalid using colors like red.		
2/20/26	0.5	Documentation	I created the Nonfunctional requirements to have the performace requirements, safety requirements, security requirements, software requirements, software quality attributes, and business rules	Next steps are to set up the development environment and start coding.	
2/21/26	0.25	Coding	I went to the spring initializer website and set up the projects metadata along with the java verison, spring version, maven build, and the dependencies.		
2/21/26	3	Coding	After I set up the spring boot project, I started writing java code for all the classes that i wrote down in my class diagram. I set up the controller and tha algorithm strategy but i did not get around to implementing any cpu scheduling algorithms.	Whats next is to implement FCFS and to create a simple frontend that accepts data.	All the classes are looking good and I am ready to start writing the algorithms.
2/23/26	6	Research, Training, Lear	I spent the day learning the basics of HTML and JavaScript and getting a hang of the languages.		I learned a lot about the basics of frontend development. I used the geeks for geeks website so I can have a structured learning path I can follow. https://www.geeksforgeeks.org/blogs/60-days-of-frontend-development/. I didnt get to learn all of it but I got up to a good part of information.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
2/25/26	3	Coding	I picked up where I left off last week and started working on the backend code. I managed to implement the FCFS algorithm that creates a ScheduleResult object with a list of GanttBlocks, list of ProcessResults, average waiting time, average turnaround time, and throughput. This was all done by going through every process in the list and simulating a FCFS algorithm and collecting the relevant data I needed. I also created a class with a method that helps me sort processes by arrival time.		Before I started coding, i sketched out a draft of what the algorithm should do on paper and was able to translate it to code. This helps me get out my ideas and not forget about them which would happen if I do it in my head.
2/27/26	1	Coding	I started writing out the HTML for the project and I was able to make the select bar where the use can select their algorithm, the add process button, run algorithm button, and a reuslts section.		
2/27/26	1	Research, Training, Lear	I did some research on how I should implement my JavaScript. I learned about DOMContentLoaded for the document event listener and I decided this would be a good fit.		I used the mozilla documentation. https://developer.mozilla.org/en-US/docs/Web/API/Document/DOMContentLoaded_event
2/27/26	3	Coding	I started writing out the javascript by getting elements from the html document and creating a create process function. This function creates html that the user will see when he clicks the add process button. It will shows the user what process you are creating, the fields for input which are arrival time and burst time, and a button to delete the process if you made a mistake.		
2/28/26	2	Coding	I continued implementing my main.js file and created event listeners for my add process button, my process container that holds all the processes, and my run simulation button.	I tried testing out the frontend and the backend together and I had a problem with getting the javascript but there was a problem becasue my frontend coudn't access my backend http endpoint.	I used https://www.baeldung.com/spring-cors to help me create a cors mapping class so my frontend and backend go to the same endpoint.
3/3/26	2	Research, Training, Lear	I was looking at different ways websites take input. I went on webistes and noticed that when you click to add something, a widget pops up and you can input things into that. I did more research and found out it has a name. They are called modals, so I went on youtube and found tutorials on adding modals to your project.	Next step is to follow the tuotial and implement modals into my project.	
3/4/26	3	Coding	I implemented the modal into my frontend and changed the htmls add process button to open a dialog which is the modal. Then i ad to update my javascript to work with the new dialog and I made sure that the user sees the process that were added when they submit a process.		
3/4/26	0.7	Research, Training, Lear	I did some reasearch on JavaDoc which is how to document java classes and I learned that I my IntelliJ IDE automatically formats the comments for me after typing /**, then it adds "@param" and "@return" if im commenting a method.	Start writing JavaDoc	

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
3/4/26	2.3	Documentation	I started documenting my java classes because that is something I have been neglecting. I wrote comments for the AlgorithmStrategy, FCFSAlgorihtm, WebConfig, SimulationController, SchedulerService, and ProcessSorter classes. For every class, i wront comments for the class and for all its methods.		
3/5/26	1.2	Documentation	I wrote the README file for my project. Since I didn't have one before, I had to add one for the peer review. I wanted my peer reviewer to undertsand what my project is and how to use it. I also included that my project uses Java 25 so in order to run my program, you have to install java 25 on your computer or it wont compile. I also included instructions on how to run the project, what directory to be in, and where the html is located to find the frontend after you run the backend. Also there are instructions on how to shop the spring boot application, and the technologies used in the project.		This is a good thing to add to my project becasue every major project on github has one so having one for my project only makes sense.
3/5/26	0.8	Documentation	I wrote documentation for my FCFSAlgorithm class. i wront JavaDoc for the class and for the runAlgorithm method in the class.		
3/5/26	0.2	Research, Training, Learn	I googled how to test java classes and methods and found out about unit testing so i decided that it will be a good fit for my project.	Add unit tests to my project to test the FCFS Algorithm implementation	
3/5/26	1	Testing & Debugging	I wanted to test my FCFSAlgorithm class so I added some unit tests to see if the algorithm outputs correct data with processes given. My algorithm passes the test so I will leave it for now. Maybe in the future, I can improve the algorithm to run more efficient if there is time.		
3/6/26	1	Research, Training, Learn	I wanted to change my frotnend to use more modern frontend technologies and I found out about typescript and React.	I will try to create a frontend using these new technologies.	
3/6/26	3	Coding	I followed a tutorial to add typescript and react to my project using vite in the terminal and I have a section for the frontend now. I wrote some components and types and the app.tsx but I am still new to these technologies so it was very confusing to me.	I couldn't get a working frontend with these technologies today so I didn't commit so I dont confuse anyone reading my repository with a broken frontend.	I need to learn how to use these technologies in more detail. I am new to frontend so I bought a course so i can learn them. I am planning to watch them in two weeks because I want to go back to working on the frontend next week.
3/9/26	1.5	Testing & Debugging	I created a Project board on github for my project. I used a template they recommended which has Backlog, Ready, In progress, In review, and Done. I also added items and moved them to the appropriate section. Now I have a working project board that will help me keep track of my tasks in my project.		I should have done this earlier but I didnt know about this until I read the peer review worlsheet
3/9/26	0.5	Other	I took time to go over my repositoy and make everything look nice, I added my documentation pdf's to my project and made sure that my peer reviewer would be able to understand how to naviagte my project.	Send an email to my peer revew partner and send him my repository	

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
3/10/26	4	Coding	I created the SJFAlgorithm class for my project. This is the algorithm that I had as the next one to implement. The implementation was very different that FCFS because I had to only work on processes that are in the ready queue at the current time.	This implementation took a long time to make because I was running into a lot of problems. trying to get the right processes into the ready list btu eventually i figured it out.	
3/11/26	2	Testing & Debugging	I wrote tests for the SJFAlgorithm implementation class so I can confirm that my implementation is working correctly. I wrote down three different tests that test for different things. One tests a refular test with basic processes. the next tests for if there is a time where there is no processes and the last one tests for how the algorithm would work if all processes start at 0.		
3/12/26	5	Coding	I wrote down the SRTFAlgorithm implementation which is the third cpu scheduling algorithm I implemented. This algorithm is much longer than the rest because now, processes can be interrupted in the middle of running for a process that has a shorter remainig burst time.	I ran into problems like the process not getting swapped out in the right time.	
3/13/26	2	Testing & Debugging	I wrote tests for the SRTFAlgorithm class that tests a basic input which it passed. I also testsed for if there is a section with no processes ready to execute and if all processes arrive at the same time.		
3/14/26	1	Research, Training, Learn	I watched one hour of video for learning typescript in a course.		
3/14/26	2.5	Other	I completed the peer review evaluation form and the google document. I also tested out the repository and tried to run it the way it was intended but I ran into problems with the database.		
3/16/26	5	Coding	I deleted the old frontend and decided to start from scratch using html, js, and css. I built the main lage layout with the algorithm selector dropdown which includes FCFS, SJF, SRTF, RR, and Priority. I also added a input for time quantum for RR. Its is not enabled yet but I have it ready to implement text after I create the RR scheduling algorithm in the backend. I also created the process table holder where users see the processes that they add. Along with that i created a new modal that adds the processes you need with arrival time, burst time, and an optional field priority. It also has input validation to make sure the processes have input in between 0 and 99 and that only numbers can be used. I also made the processes as cards where they can render and you can delete them.	Finish implementing the frontend JS and CSS	I feel more confident with this approach Im using now. I feel like i set myself up in a way where I can easily add new functions without getting too complicated.
3/17/26	6	Coding	I set up the connection between the frontend and the backend server using localhost 8080. I built the renderer for the results which uses the backends response. It creates a gantt chart with the time it starts and ends. I also implemented idle blocks to respresent when the cpu isnt being used so it is in idle mode. I also implemented the process result table showing the processes waiting time, turnaround time, arrival, burst and priority. I also added a summary which shows the average waiting time, average turnaround time, and the throughput.		The frontend is functioning and now i need to add styles to make it look better.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
3/18/26	4	Design	I wasworking on the CSS. I added body padding and background color to make the look feel consistent across all the sections. I used flex boxes to properly align the algorithm selector bar with the correct space between the dropdown, quantum input, and run button. I styled the process cards with padding, border styling and color and made the text visible. I styled the odal with box size, button styles for add and cancel. I made the gantt chart clear with block heights, font size, borders, and the idle block having a different style.I styles the results section to make it look cleaner with the table and the summary.		Looks a lot better than the last version of the frontend. It also has more functionality.
3/26/26	6	Testing & Debugging	I wrote more unit tests for the FCFS, SJF, and SRTF algorithms to cover more ground. I also added documentation for all the test cases I made because i saw that previously, my documentation for tests were weak because I didnt spend a lot of time on them. For fcfs, the tests covered the sorting of the arrival order and handling idle gaps. There was also handling irraval time at zero. For SJF, the tests validates that shorter jobs jump ahead of longer waiting ones. I tested the tie breaking by arrival time when burst times are equal. idle gaps, and maximum and minimum burst times to test if everything is working. For SRTF, i tesed the preemption to make sure every process was interrupted when it needed to be and which one needs to go next. I also tested where no preemption occurs. I tested the gantt charts, tie breakers, and idles.	Now i need to work on the Final Slides.	Caught some bugs and fixed them so now I can say that my algorithms work better now.
3/27/26	1	Other	I lauched my project to the internet so anyone can use it without downloading the repository and setting it up. I hosted my backend on the railway website for free and connected it to the frontend by switching the js fetch.		https://j-koral.github.io/cpu-scheduling-simulator/
3/27/26	3	Other	Started working on the final draft for the slides. I started by making a copy of my initial draft and reviewing the contents. I decided that there was not a lot of text on the slides so I decided that that was a starting point. I added more details on the first 7 slides and provided details for the wireframe. I also added a new slide that shows the github pages of the website with it in use.		
3/28/26	0.01	Supervisor Discussion	I sent an email to Minh Le about my progress with the project and shared him the link to my github pages so he can test out the functionality. I also gave him the Supervisor Evaluation Sheet with the first page filled out.		
3/28/26	3	Other	I wrote commentary for the rest of the diagram slides so people who don't know what they are looking at can read the text and understand what the diagrams are showing. I added three new slides, the scheduling algorithms design and efficency slide, Toughest bug slide, and testing in Junit slide. I gave detailed discription of each algorithm i implemented and their runtime costs and why. I explained the toughest bug i had to deal with in SRTF, and I gave a general summary of the tests that I did and what they covered.	I have a good base for the slides, but i still need to finish off a few more slides and add styles to the slides instead of boring white slides.	

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
3/29/26	2	Other	I finished off the slides by remodleing the three slides that hd the tentative schedule and transformed it into one slide summarzing what I completed and what I still have planned to do. I also added the project links slide and put links to my github repository, github pages, project management board, and project documents. I also styled all of the slides to make them look presentable. I aligned each text boxes, gave them boarders, put a theme on the slides and wroked on positioning things to fit in the slides.	I need to start working on implementing the RR algorithm.	
3/31/26	2	Research, Training, Learn	I started out the process of implemeting the RoundRobin algorithm. I learned from my mistakes during the last couple of implementations and decided it was best to do some research on Round Robin preemption logic and the role of the time quantum so that I can more accurately replicate the process in java.	Write the java implemenation of RR	
4/1/26	4	Coding	I implemented the skeleton for the RR algorithm. I used the ArrayDeque as the ready queue so that we can have O (1) efficiency for the processes. We start by verifying the readyqueue is empty and if it is, then it advances the current time to match the next processes arrival time. After polling the next process from the queue, we calculate the runtime slice by taking the minimum of the processes ramining burst time and the quantum time. The simulation time is then advanced by the slice and we generate the gantt chart entry for the specific process that ran and for how long. Then we check for any new processes that arrived during the execultion and add them to the ready queue. Then we check if the current process still has a burst time greater than 0 and if it does, we put it back in the queue. The rest is similar to the other scheduling algorthm implementation where we organize the results to send back.		I feel very confident with my implementation but I still need to do real testing to make sure I didnt miss anything when implementing.
4/2/26	5.5	Testing & Debugging	I created the testing class for RR to find any bugs in my implementation. I spent a lot of time debugging the moment where a process is re-added to the process queue to ensure that the new arrivals during a time slice are given priority over a process that got preempted. I had to do a lot of planning on paper and tracing my old code to figure out where things went wrong and to help plan how to fix them.		
4/3/26	3.5	Coding	I was focusing on improving the algorithm when the CPU is idling. I noticed that the algorithm failed when the first process arrived after time 0 so i added a loop that advances the simulation clock to the next available time. I also picked up a nice trick I was the other day where I should create a copy of a list instead of manupilating the original passed list.		

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
4/4/26	1.5	Testing & Debugging	When i tried to use the algorithm combined with ym frontend, I was getting a problem in the IDE. It turns out that I forgot that my Scheduler Service method getAlgorithmKey returns "RR" to help find th java class but my class was not named RR, it was named RoundRobin so I changed the name of my class to RRAlgorithm to keep things consistent.	Next, I Should work on the frotnend and implement new desig so the user feels better when using the simulator.	
4/6/26	4.5	Research, Training, Learn	I did some resarch on UI design by watching a bunch of youtube videos and I wrote down notes on thing I think would be nice to add for the design to my frontend. Things like coloring the processses with different colors, animate the gantt chart, an algorithm comparison mode, stats cards, transition animation when running a new algorithm, and a tap for data on te gantt chart blocks to make it interactive.		
4/7/26	3.5	Design	I implemented process color coding. Each process added should be distiguished by its own color. I defined an array for 8 colors in main.js and a processColorMap object that maps each process id to a color. When a process is added, it gets assigned a color index based on the position in the couter. The color index is then used in the CSS class so that the same process always appears in the same color in the process holder, ganthcart, and process table. Now it is very clear to see where the process is in the gantt chart and the table whereas before, everything eas blue and you couldnt tell them apart besides the label.		
4/9/26	2.5	Design	I implemented the color coded stat cards. I redesigned the averages section at the bottom of the results to use color coded stat cards. Instead of just dispaying the raw numbers, each card will evaluate the performance of the processes with the scheduling algorithm and change the color to green, amber, or red depending on whether the reuslt is good, mdoerate, or poor.		
4/11/26	4	Design	I implemented the tool tips function to give users more detail about each process when hovering your mouse over the gantt chart. I added tool tips to every gantt chart entry that shows the process id, the time range it ran, its duration, and how long it waited to run.		
4/12/26	1.35	Other	The server I was hosting my backend from was shut down because of usage. I tried renderer but i still had no success linking it to github pages.	I need to find a free server hoster that will host my beckend for me for free.	
4/14/26 - 4/15/26	8	Coding	I implemented the PriorityAlgorithm class for my project. The implementation follows the same non-preemptive structure as SJF but instead of sorting the ready queue by burst time, I sorted it by priority number where a lower number means higher priority. I used a Comparator that breaks ties by arrival time so that if two processes share the same priority, the one that arrived earlier runs first. The rest of the logic for building the GanttEntry list, ProcessResult list, and calculating the averages and throughput were mostly the same as the other algorithms.	Now I need to write tests to make sure the algorithm is working properly for edge cases.	The last algorithm is finally implemented but still needs proper testing.

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
4/16/26	6	Testing & Debugging	I wrote the test class for the PriorityAlgorithm. I covered senarios like a high priority process jumping ahead of a lower priority one, four processes queuing up behind a long running job and then being dispatched in priority order, all processes arriving at time 0 and running in priority order, ties in priority being broken by arrival time, a single process, an idle CPU gap between two processes, priority 0 being treated as the highest priority, and per-process waiting and turnaround time verification.		Now that the algorithm is fully tested and working, we can move on to the next feature which will probably be feature to run multiple algorithms at the same time.
4/17/26	2	Documentation	I worked on the project slides and made a few updates to the style of the slides. Since the app isnt finished yet, there is not much else to add that isnt a final product.		
4/20/26	2	Coding	I implemented the algorithm comparison feature. I added two new DTO classes, ComparisonRequest and ComparisonResult. ComparisonRequest holds a list of algorithm names, an optional quantum for RR, and the shared process list. ComparisonResult wraps a LinkedHashMap that maps each algorithm name to its ScheduleResult so the frontend receives all results in one response and in the order they were requested.		
4/21/26	5	Coding	I added the runComparison method to the SchedulerService. This method loops over the requested algorithm names, gets each one to its strategy using a new private resolveStrategy helper, runs the algorithm on a deep copy of the process list, and stores the result in the map. I made sure to deep copy the process list for each algorithm because some algorithms sort or mutate the list in place and I didnt want one algorithm's run affecting anothers. I also added a POST /api/compare endpoint to the SimulationController that calls this new method and returns the ComparisonResult with an HTTP 200.	Now i have to update the frontend to accomodate for this new feature.	
4/22/26	3.5	Coding	I updated the frontend to use the new comparison feature. I added a Single and Compare mode toggle button to the algorithm selector bar. In Single mode, the existing dropdown behavior is the same. In Compare mode, the dropdown is replaced with a row of checkboxes, one per algorithm, so the user can select multiple algorithms at once. The quantum input shows and hides automatically depending on whether RR is checked. I also updated the run button to dispatch to either runAlgorithm or runComparison depending on the active mode.		
4/23/26	5.5		I implemented the comparison results section in the frontend. When the compare mode results come back from the backend, the frontend renders a summary table with one row per algorithm showing the average waiting time, average turnaround time, and throughput side by side. The best value in each column gets a green winner highlight with a trophy emoji so it is easy to see which algorithm performed best on that workload of processes. Below the summary table, each algorithm gets its own collapsible section with a full Gantt chart and process table so the user can go into the details if they want to.	Next we have to fix up our frontend code and separate them into multiple files instead of static.css and main.js	

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
4/27/26	7.5	Design	I refactored the frontend because there was a bunch of code only in two files and i changed it into a modular structure using ES modules. The old main.js and styles. css were deleted and replaced with 7 JS files and 8 CSS files. For JavaScript, I created state.js for the shared processes array and color helpers, modal.js for the add and delete process logic, mode.js for the Single and Compare toggle, api.js for all fetch calls, render.js for the shared Gantt chart and process table builders plus the single mode rendering, compare.js for the comparison summary table and collapsible per-algorithm sections, and a new main.js that only handles wiring all event listeners on DOMContentLoaded. For CSS, I split the styles into base.css, layout.css, modal.css, process.css, gantt.css, table.css, averages.css, and compare.css. I updated index.html to link all eight CSS files and load the JS entry point with type="module" which enables native ES imports without needing any build tools since Spring Boot already serves the static files over HTTP. All inline onclick handlers were also removed from the HTML because we used addEventListener calls in main.js.		Looks much nicer now and more easy to read. It is very organized but it took a lot of energy to accomplish.
4/28/26	1.5	Coding	I added a random process generation feature to the frontend. A count input and a Generate Random button were added below the Add Process button in the process container. Clicking the button generates the specific number of processes with randomized arrival times between 0 and 14, burst times between 1 and 10, and priority values between 1 and 10. Priority is always assigned a value so that no process defaults to 0, which was the previous behavior when the priority field was left blank.		I should have done this sooner because before i was namually typing in processes to test functionality.
4/29/26 - 4/30/26	9.5	Coding	I implemented three themes for the user to pick from for the app. A new theme.css file defines CSS custom properties for all colors across 3 themes: Light, Classic Dark, and Dark Blue. All 8 CSS files were updated to use these variables instead of hard-coded color values. A new theme.js module handles reading and writing the active theme to localStorage so the user's preference persists between sessions. A fixed theme switcher widget was added to the top right corner of the page with three buttons to switch between themes instantly.	It was very annoying to go through all the css and find specific things that needed to be changed with the new variables.	
5/4/26	4.5	Coding	I implemented a sidebar that holds recent runs. A hamburger button fixed to the top left of the page opens the sidebar with a slide-in animation. Each time an algorithm is run, the run is saved to localStorage with the algorithm name, mode, process count, timestamp, and result statistics. In single mode, each entry also displays the average waiting time, average turnaround time, and throughput as small stat pills. Timestamps are shown as relative time such as "5m ago" or "2h ago". The sidebar holds up to 20 entries and includes a Clear All button at the bottom.		

Date	Duration (hours)	Category	Description of completed task	Challenges and/or next steps	Reflection
5/5/26	4	Coding	I added more functionality to the sidebar so that clicking any entry replays that run automatically. When a recent entry is clicked, the full process list that was active during that run is restored from the saved snapshot in localStorage, the process cards in the UI are cleared and remade, the algorithm selector or compare checkboxes are set to match the original run, and the algorithm is executed.		Great feature that makes this app stand apart because now you could re-run old processes you did again without having an account.
5/7/26	3	Documentation	I finished editing the slides adding up to date features of what the final version of the app provides.		
5/8/26	3.5	Other	I re-delopyed the app. There was a lot of problems going through this process trying to deploy my backend for free. I ran out of usage last time so I copied my repository on a new github account. I deployed the app there and spent a lot of time trying to get the github pages to work correctly with the backend. the server on Render was have a lot of issues deploying so it took me a while to get it working.		https://j-koral.github.io/cpu-scheduling-simulator/ The app is deployed and usable. It may take around 1 minute for the first run to go through due to the server spinning down when innactive.